

Introduzione all'Algoritmo ELO

lgoritmo ELO è uno dei metodi più diffusi per classificare i giocatori in giochi da tavolo, sport e videogiochi competitivi. Il sistema assegna un punteggio (rank) a ciascun giocatore e lo aggiorna dopo ogni partita, in base al risultato ottenuto rispetto a quello atteso.

Definizioni principali

- r_i : punteggio (rank) del giocatore i
- S_{ij} : esito della partita tra giocatore i e giocatore j
 - $S_{ij} = 1$ se vince i
 - $S_{ij} = 0$ se vince j
 - $S_{ij} = \frac{1}{2}$ in caso di pareggio
- S_{ji} : vale $1 - S_{ij}$

Il cuore del metodo ELO è il confronto tra il **risultato reale** e il **risultato atteso**.

Risultato atteso

Il risultato atteso del giocatore i contro il giocatore j è dato da una funzione di probabilità:

$$\mu_{i,j} = \frac{1}{1 + 10^{\frac{r_j - r_i}{\zeta}}} \qquad \mu_{i,j} = \frac{10^{r_i/\zeta}}{10^{r_i/\zeta} + 10^{r_j/\zeta}}$$

dove:

- r_i e r_j sono i rank attuali dei due giocatori,
- ζ è un parametro di scala (tipicamente $\zeta = 400$).

Aggiornamento del punteggio

Dopo ogni partita, i punteggi di entrambi i giocatori vengono aggiornati.
Il nuovo rank è dato da:

$$r_i \leftarrow r_i + K (S_{i,j} - \mu_{i,j})$$
$$r_j \leftarrow r_j + K (S_{j,i} - \mu_{j,i})$$

dove:

- K è la costante di aggiornamento (K-factor), che controlla la sensibilità del sistema.
- $r_0 = 0$ per tutti i giocatori all'inizio del torneo.

Analisi Preliminare

Grafico vittore di ogni giocatore

```
# aggiungo una colonna vincitore = (id-white se Score=1, id-black se Score=0 altrimenti p )

vincitori <- data %>%
  mutate(
    id_winner = case_when(
      Score == 1 ~ White, # se Score = 1 -> vince il bianco
      Score == 0 ~ Black, # se Score = 0 -> vince il nero
      TRUE ~ -1          # altrimenti pareggio
    ),
    col_winner = case_when(
      Score == 1 ~ "W", # se Score = 1 -> vince il bianco
      Score == 0 ~ "B", # se Score = 0 -> vince il nero
      TRUE ~ "P"       # altrimenti pareggio
    )
  )

head(vincitori)
```

##	White	Black	Score	id_winner	col_winner
## 1	73	1246	0.5	-1	P
## 2	73	5097	0.5	-1	P
## 3	73	5104	0.5	-1	P
## 4	73	7321	1.0	73	W
## 5	73	7375	0.5	-1	P
## 6	88	1033	0.5	-1	P

Barplot Vittoriori

```
vittorie = vincitori %>%
  group_by(id_winner) %>%
  summarise(num_vittorie = n()) %>%
  filter(id_winner != -1) %>%
  arrange(-num_vittorie)
```

top20 <- vittorie %>%
 arrange(desc(num_vittorie)) %>%
 slice(1:20)

```
# Barplot
ggplot(top20, aes(x = reorder(factor(id_winner), -num_vittorie),
  y = num_vittorie)) +
  geom_col(fill = "steelblue") +
  labs(
    title = "Top 20 giocatori per numero di vittorie",
    x = "ID Giocatore",
    y = "Numero di vittorie"
  ) +
  theme_minimal() +
  theme(axis.title.x = element_text(angle = 90, hjust = 1))
```

I BIANCHI HANNO UN VANTAGGIO?

```
vincitori %>%
  count(col_winner)
```

##	col_winner	n
## 1	B	15224
## 2	P	28666
## 3	W	21163

```
ggplot(data = vincitori) +
  geom_bar(mapping = aes(x = col_winner))
```

```
num_partite = nrow(vittorie)
```

Sembraerebbe di si, ma bisognerebbe testare tramite INFERENZA STATISTICA
fare un binomial

CALCOLIAMO IL RANK DEI VARI GIOCATORI

verifichiamo la lista dei giocatori

```
giocatori <- sort(unique(c(vincitori$White, vincitori$Black))) # lista id giocatori
# potrebbero esserci id mancanti

last_id = giocatori[length(giocatori)] # ultimo id rappresentato

mancanti <- setdiff(1:last_id, giocatori) # sono tutti gli id mancanti
```

Calcoliamo il rank di ogni giocatore

```
# Partono tutti da 0

# I giocatori mancanti rimarranno con rank 0

# i-esima posizione è il rank del id = i giocatore

RANK = rep(0, last_id)
```

Per ogni partita, vengono aggiornati i rank

```
# Ri = Ri + (Si - uj)

head(vincitori)
```

##	White	Black	Score	id_winner	col_winner
## 1	73	1246	0.5	-1	P
## 2	73	5097	0.5	-1	P
## 3	73	5104	0.5	-1	P
## 4	73	7321	1.0	73	W
## 5	73	7375	0.5	-1	P
## 6	88	1033	0.5	-1	P

```
# FUNZIONE CHE VIENE ESEGUITA PER OGNI RIGA DI vincitori
# (cioè per ogni partita)

# risultato atteso Elo: mu_ij = P(i batte j)
elo_expected <- function(ri, rj, zeta = 400) {
  1 / (1 + 10^((rj - ri) / zeta))
  # equivalente: 10^(ri/zeta) / (10^(ri/zeta) + 10^(rj/zeta))
}

aggiorna_rank_elo <- function(vincitori, RANK, K = 25, zeta = 400) {

  for (k in seq_len(nrow(vincitori))) { # per ogni k-esima riga di vincitori
    i <- vincitori$White[k]              # prendo il i = bianco
    j <- vincitori$Black[k]              # prendo il j = nero
    if (is.na(i) || is.na(j)) next

    Ri <- RANK[i]; Rj <- RANK[j]        # prendo il rank di i e di j

    # estraggo gli SCORE REALI

    sij <- vincitori$Score[k]
    sji <- 1 - sij

    # calcolo gli SCORE ATTESI

    muij <- elo_expected(Ri, Rj, zeta)
    muij <- 1 - muij

    # AGGIORNO IL RANK DEI 2 GIOCATORI

    RANK[i] <- Ri + K * (sij - muij)
    RANK[j] <- Rj + K * (sji - muij)
  }

  RANK
}

RANK_updated = aggiorna_rank_elo(vincitori, RANK)
```

```
df_rank <- data.frame(
  id = seq_along(RANK_updated),
  rank = round(RANK_updated, 3) # arrotonda a 3 decimali
)

head(df_rank)
```

##	id	rank
## 1	1	101.770
## 2	2	-1.811
## 3	3	-20.048
## 4	4	-10.192
## 5	5	-43.469
## 6	6	12.568

sum(df_rank\$rank)

```
## [1] 0.012
```

RANK A FINE TORNEO

```
library(dplyr)
library(ggplot2)

# prendo i primi 20 ordinati per rank
df_top20 <- df_rank %>%
  arrange(desc(rank)) %>%
  slice(1:20)
```

```
ggplot(df_top20, aes(x = reorder(as.factor(id), rank), y = rank)) +
  geom_col(fill = "steelblue") +
  geom_text(aes(label = rank),
    position = position_stack(vjust = 0.5), # al centro della barra
    color = "white", fontface = "bold") +
  coord_flip() +
  labs(title = "Classifica Top 20 Giocatori",
    x = "ID Giocatore",
    y = "Rank") +
  theme_minimal()
```

Come sono distribuiti i ratings?

```
library(ggplot2)
library(gridExtra)

##
## Attaching package: 'gridExtra'

## The following object is masked from 'package:dplyr':
##
##   combine

# calcolo media e deviazione standard del campione
mu <- mean(df_rank$rank)
sigma <- sd(df_rank$rank)

# grafico 1: istogramma + curva normale
p1 <- ggplot(df_rank, aes(x = rank)) +
  geom_histogram(aes(y = ..density..), binwidth = 0.8,
    fill = "steelblue", color = "white", alpha = 0.8) +
  stat_function(fun = dnorm, args = list(mean = mu, sd = sigma),
    color = "red", size = 1) +
  coord_cartesian(xlim = c(-5, 10)) +
  labs(title = "Distribuzione dei Rank con curva Normale",
    x = "Rank",
    y = "Density") +
  theme_minimal()
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
# grafico 2: boxplot zoomato
p2 <- ggplot(df_rank, aes(x = factor(1), y = rank)) +
  geom_boxplot(fill = ..density.., alpha = 0.6, outlier.shape = NA) +
  coord_cartesian(ylim = c(-1.2, 0.2)) +
  labs(title = "Boxplot dei Rank",
    x = "",
    y = "Rank") +
  theme_minimal()
```

```
# metto i due grafici affiancati
grid.arrange(p1, p2, ncol = 2)
```

```
## Warning: The dot-dot notation ('..density..') was deprecated in ggplot2 3.4.0.
## i Please use `after_stat(density)` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

Analisi della distribuzione dei Rank

Dall'osservazione dei due grafici si nota che:

- La distribuzione è centrata attorno allo zero (media = 0).
- L'istogramma mostra una concentrazione molto forte vicino a 0, più appuntita rispetto a una normale con stessa media e varianza → la distribuzione è **leptocurtica**.
- La curva normale stimata appare più piatta e larga, a causa della presenza di outlier che gonfiano la deviazione standard.
- Le code risultano più pesanti rispetto a una gaussiana, come confermato anche dal boxplot.
- In sintesi, la distribuzione non è perfettamente normale: ha un picco centrale più alto e code più lunghe.

DIMOSTRARE LA PROPRIETA'

" La somma dei ratings di tutti i giocatori è uguale a 0 "

```
sum(df_rank$rank)
```

```
## [1] 0.012
```

Proprietà di conservazione della somma dei punteggi Elo

Una proprietà fondamentale del sistema Elo è che la **somma di tutti i punteggi dei giocatori rimane costante** (nel nostro caso è pari a 0).

Dimostrazione formale

Dato un match tra due giocatori i e j :

- Il punteggio osservato per i è $s_{i,j}$, mentre per j è $s_{j,i} = 1 - s_{i,j}$.
- Il punteggio atteso per i è $\mu_{i,j}$, mentre per j è $\mu_{j,i} = 1 - \mu_{i,j}$.

L'aggiornamento Elo è:

$$r'_i = r_i + K \cdot (s_{i,j} - \mu_{i,j})$$
$$r'_j = r_j + K \cdot (s_{j,i} - \mu_{j,i})$$

Sommandoli:

$$r'_i + r'_j = r_i + r_j + K \left[(s_{i,j} - \mu_{i,j}) + (s_{j,i} - \mu_{j,i}) \right]$$

Poiché $s_{i,j} + s_{j,i} = 1$ e $\mu_{i,j} + \mu_{j,i} = 1$:

$$r'_i + r'_j = r_i + r_j + K \cdot (1 - 1) = r_i + r_j$$

Quindi la **somma rimane invariata** ad ogni partita.

Se i punteggi iniziali hanno somma zero, la somma resterà sempre zero.

Modifico i parametri zeta = 400 e k = 25

```
RANK_parametri = aggiorna_rank_elo(vincitori, RANK, 25,400)

# costruisco il dataframe dei rank aggiornati
df_rank_param <- data.frame(
  id = seq_along(RANK_parametri),
  rank = RANK_parametri
)

# istogramma della distribuzione
ggplot(df_rank_param, aes(x = rank)) +
  geom_histogram(binwidth = 20, fill = "steelblue", color = "white", alpha = 0.8) +
  labs(title = "Distribuzione dei Rank con ζ=400, K=25",
    x = "Rank",
    y = "Frequenza") +
  theme_minimal()
```

Cambia solo la velocità degli aggiornamenti del rank MA L'ISTOGRAMMA RIMANE UGUALE

CORRELAZIONE TRA I PUNTI OTTENUTI E IL RATING

Calcolo la somma dei punteggi

```
vincitori = data %>%
  mutate(
    id_winner = case_when(
      Score == 1 ~ White, # se Score = 1 -> vince il bianco
      Score == 0 ~ Black, # se Score = 0 -> vince il nero
      TRUE ~ -1          # altrimenti pareggio
    ),
    col_winner = case_when(
      Score == 1 ~ "W", # se Score = 1 -> vince il bianco
      Score == 0 ~ "B", # se Score = 0 -> vince il nero
      TRUE ~ "P"       # altrimenti pareggio
    )
  )

df_punteggi = vincitori %>%
  filter(id_winner != -1) %>%
  group_by(id_winner) %>%
  summarize(somma_punti = sum(Score)) %>%
  arrange(-somma_punti)

head(df_punteggi)
```

##	# A tibble: 6 × 2	
##	id_winner	somma_punti
##	<dbl>	<dbl>
## 1	4850	67
## 2	1594	62
## 3	4037	62
## 4	1286	59
## 5	64	52
## 6	1098	51

Unisco punteggi e rank

```
punteggi_rank <- inner_join(df_rank, df_punteggi, by = c("id" = "id_winner"))

# Correlazione
cor(punteggi_rank$rank, punteggi_rank$somma_punti)
```

```
## [1] 0.7257161
```

Forte correlazione positiva, quindi si sono correlati

Verifichiamo se è presente correlazione tra numero partite giocate e punteggio

estralimo il numero di partite giocate per ogni giocatore

```
df_num_games <- bind_rows(
  vincitori %>% select(id = White),
  vincitori %>% select(id = Black)
) %>%
  group_by(id) %>%
  summarise(num_games = n(), .groups = "drop") %>%
  arrange(desc(num_games))

head(df_num_games)
```

##	# A tibble: 6 × 2	
##	id	num_games
##	<int>	<int>
## 1	4850	280
## 2	1594	276
## 3	4037	272
## 4	1286	270
## 5	64	258
## 6	1512	257

E verifichiamo la correlazione

```
df_corr <- df_rank %>%
  left_join(df_num_games, by = "id")

# calcolo la correlazione tra numero partite ed Elo
cor_elo <- cor(df_corr$rank, df_corr$num_games, use = "complete.obs")

cor_elo
```

```
## [1] 0.6387939
```